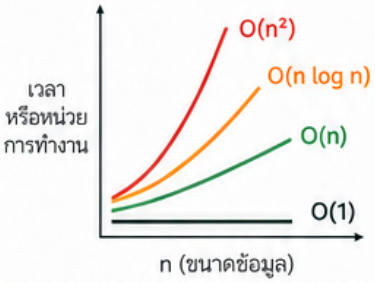
Data Structures & Algorithms

Dev Trail · ภาพรวมสำหรับทบทวน

1 Big-O และการวิเคราะห์ความซับซ้อน

Big-O คือขอบบนของการเติบโตเมื่อ n โต
ตัดค่าคงที่และพจน์ย่อยทิ้ง และต้องบอกว่าหมายถึง worst, average หรือ best case

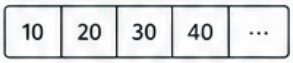


การระบุกรณี

- worst case : แย่ที่สุด (ขอบบนที่รับประกันได้)
- average case : โดยเฉลี่ย
- best case : ดีที่สุด (กรณีที่เป็นไปได้)

2 โครงสร้างข้อมูลพื้นฐาน

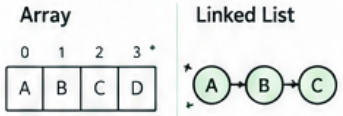
Dynamic Array (เช่น Array)



- push ท้ายเป็น amortized O(1) เพราะขยายเป็นสัดส่วน
- shift ต้องเลื่อนทุกตัว จึงไม่ควรใช้ทำคิวยาว

⚠ ไม่ควรใช้ shift ทำคิวยาว

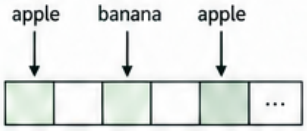
Array vs Linked List



เข้าถึง index ได้ O(1) และใช้ cache ได้ดี

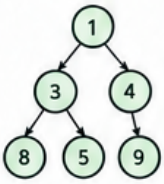
แทรกได้ O(1) เฉพาะเมื่อติอโหนดตรงนั้นอยู่แล้ว

Hash Table



- เฉลี่ย O(1) แต่ worst case O(n) เมื่อ key ชนกัน
- ต้อง resize ตาม load factor
- ระวัง hash flooding

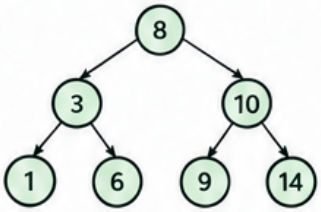
Heap (Min-Heap)



- ดูค่าน้อยสุด O(1)
- insert / pop O(log n)
- สร้างจาก array O(n)
- top-k ใช้ heap ขนาด k ได้ O(n log k)

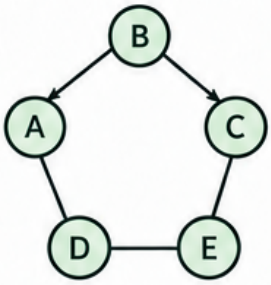
3 Tree และ Graph

Binary Search Tree (BST)



- เฉลี่ย O(log n) แต่เอียงเป็น O(n) เมื่อใส่ข้อมูลเรียง
- in-order ของ BST ได้ค่าเรียงจากน้อยไปมาก

Graph



กราฟแทนการใช้ adjacency list มีความซับซ้อน O(V + E)

BFS	ได้ทางที่ผ่านเส้นน้อยสุด (จำนวนเส้นน้อยที่สุด)
Dijkstra	ใช้เมื่อน้ำหนักไม่ติดลบ
Bellman-Ford	ใช้เมื่อมีน้ำหนักติดลบ

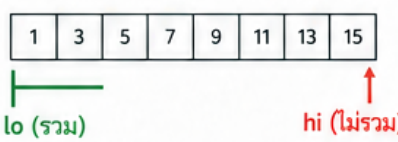
4 การจัดเรียงข้อมูล (Sorting) และการค้นหา

Comparison-based Sorting

การจัดเรียงโดยใช้การเปรียบเทียบ มีขอบล่างกรณีแย่สุด Ω(n log n)

อัลกอริทึม	Best / Average	Worst	คุณสมบัติ
Quicksort	O(n log n)	O(n²)	โดยทั่วไปไม่ stable
Merge sort	O(n log n)	O(n log n)	stable แต่ใช้ที่เพิ่ม O(n)

Binary Search ใช้ช่วงแบบ [lo, hi) คู่กับ while (lo < hi)



ใน JavaScript อย่าหาค่ากลางด้วย >>1 เมื่อค่าเกิน 2^31 − 1

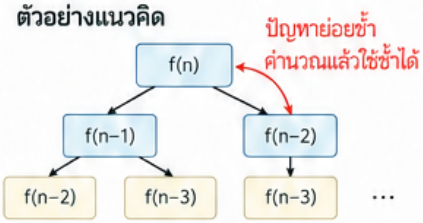
```
mid = lo + Math.floor((hi − lo) / 2)
```

5 Dynamic Programming และเทคนิคที่เกี่ยวข้อง

Dynamic Programming (DP)

ใช้เมื่อมี optimal substructure และปัญหาย่อยซ้ำ

ตัวอย่างแนวคิด



แนวทางการทำ DP

1. ระบุปัญหาย่อย
2. นิยามความสัมพันธ์
3. ใช้ memoization หรือ tabulation

Sliding Window

ใช้เทคนิคแบบขยาย-หด (two pointers) อาศัยว่าทุกค่าเป็นบวก



ใช้ได้ดีในปัญหาที่ค่าทุกค่าเป็นบวก เนื่องจากเมื่อขยายแล้วผลรวมจะไม่ลดลง

Networking & HTTP

Dev Trail · ภาพรวมสำหรับทบทวน

1 ภาพรวมลำดับการทำงานของ HTTP Request

ลำดับของ request ดูได้ในแท็บ Timing ของ DevTools

1. **DNS** — แปลงชื่อโดเมนเป็น IP
2. **TCP handshake** — เชื่อมต่อด้วย three-way handshake (กรณีใช้ TCP)
3. **TLS handshake** — ยืนยันตัวตนและสร้างการเข้ารหัส (กรณี HTTPS)
4. **HTTP request** — ส่งคำขอ เช่น GET, POST
5. **TTFB (Time to First Byte)** — รอจนได้ไบต์แรกจากเซิร์ฟเวอร์
6. **Download** — ดาวน์โหลดข้อมูลส่วนที่เหลือ

2 DNS, TCP/UDP และ TLS

ต่างหน้าที่ ต่างคุณสมบัติ สำคัญต่อการเชื่อมต่อเว็บ

DNS

- DNS TTL คือเวลาที่ resolver cache คำตอบได้
- ลด TTL ก่อนย้าย IP ไม่อย่างนั้นผู้ใช้งานส่วนจะไปที่เดิมจนกว่า cache หมดอายุ

TCP vs UDP

TCP	UDP
• reliable, in-order • เปิดด้วย three-way handshake • เหมาะกับงานที่ต้องการความถูกต้องของข้อมูล	• ไม่มีประกันการส่งถึง • ไม่รับประกันลำดับ • ไม่ต้องมี handshake • ใช้ในงานที่ยอมรับการสูญหายได้

พอร์ตเริ่มต้นของ HTTP: 80 และ HTTPS: 443

TLS (HTTPS)

- ให้ server authentication, confidentiality, integrity
- ไม่ซ่อน IP ปลายทางและขนาดข้อมูล

⚠ การใช้ -k/--insecure ทำลายการยืนยันตัวตน จึงไม่ควรใช้ในงานจริง

3 HTTP: Method, Status Code และ Redirect

เข้าใจความหมาย เพื่อออกแบบและจัดการการใช้งานให้ถูกต้อง

HTTP Method

ประเภท	Method
safe (อ่านอย่างเดียว)	GET, HEAD, OPTIONS
idempotent (ทำซ้ำได้ผลเดิม)	safe + PUT, DELETE
ไม่รับประกัน idempotency (ต้องตรวจผลซ้ำก่อน retry)	POST, PATCH

Status Code ที่พบบ่อย

401	ไม่มี credential ที่ใช้ได้ (ต้องมี WWW-Authenticate)
403	เข้าใจคำขอ แต่ปฏิเสธ
404	ใช้ซ่อน resource ที่ห้ามเข้าได้
409	ขัดสถานะ (conflict)
422	เนื้อหาใช้ไม่ได้
429	ถูกจำกัดการใช้งาน ควรดูค่า Retry-After

Redirect

301	อาจเปลี่ยน POST เป็น GET
302	อาจเปลี่ยน POST เป็น GET
307	ห้ามเปลี่ยน method
308	ห้ามเปลี่ยน method
303	พาไป GET

4 Caching และ CORS

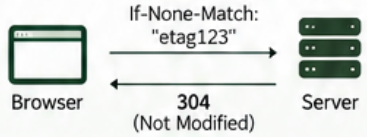
ช่วยลดการโหลดซ้ำ และควบคุมการเข้าถึงข้าม origin อย่างปลอดภัย

Cache-Control และ Conditional Request

no-store	ห้ามเก็บ cache
no-cache	เก็บได้ แต่ต้อง revalidate ทุกครั้ง
private	ห้าม shared cache (เช่น CDN)

ETag + If-None-Match

ถ้าเนื้อหาไม่เปลี่ยน → ตอบกลับ 304 (Not Modified)



CORS (Cross-Origin Resource Sharing)

- CORS บังคับโดยเบราว์เซอร์ ไม่ได้กัน curl
- request ที่ไม่ simple มี preflight OPTIONS
- ส่ง credential (เช่น cookie) ห้ามใช้ Allow-Origin: *



5 HTTP/2, HTTP/3 และการ Retry

เลือกใช้ให้เหมาะกับลักษณะงาน และจัดการการ retry อย่างถูกต้อง

HTTP/2 vs HTTP/3

HTTP/2	HTTP/3
• multplex หลาย request บน TCP เดียวกัน • ยังติดปัญหา TCP head-of-line blocking	• ใช้ QUIC บน UDP • ช่วยลดปัญหา head-of-line blocking ของ TCP

การ Retry

- ควร retry เฉพาะ error ชั่วคราว (เช่น เครือข่ายล้มเหลวชั่วคราว)
- ใช้ backoff + jitter เพื่อลดการกดโหลดซ้ำ และเพิ่มโอกาสสำเร็จ



ภาพรวมคัดประเด็นสำคัญ · อ่านรายละเอียดครบในบทเรียน Dev Trail

System Design Fundamentals

Dev Trail · ภาพรวมสำหรับทบทวน

1 เริ่มจาก Requirement และการประมาณขนาด

กำหนดความต้องการให้ชัดเจน

Functional requirement
- ระบบต้องทำอะไรได้บ้าง
- ขอบเขตของฟีเจอร์

Non-functional requirement
- วัดได้ เช่น latency, availability
- ความปลอดภัย, ความสามารถในการขยาย

จดสมมติฐาน
- ระบุสมมติฐานให้ผู้ชัดเห็น
- เช่น ปริมาณผู้ใช้, อัตรา peak, พฤติกรรมการใช้งาน

การประมาณแบบลำดับขนาด (Order of Magnitude)

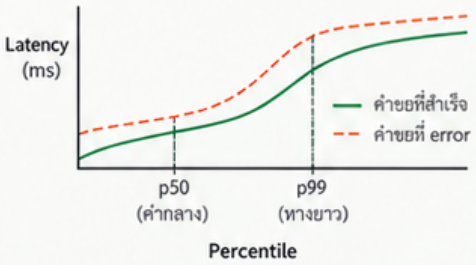
1 วัน ≈ 10^5 วินาที
(ใช้สำหรับการคำนวณคร่าว ๆ)

ตัวคูณ peak
เป็นสมมติฐาน ขึ้นกับลักษณะการใช้งาน

ตาราง latency ของ Norvig
ใช้เทียบลำดับขนาดเท่านั้น
ไม่ใช่ค่าที่ใช้กับทุกระบบ

ดู Latency แบบ percentile

ดูแนวโน้มการตอบสนองจาก percentile
ไม่ได้ค่าเฉลี่ย และแยกกรณีสำเร็จกับ error



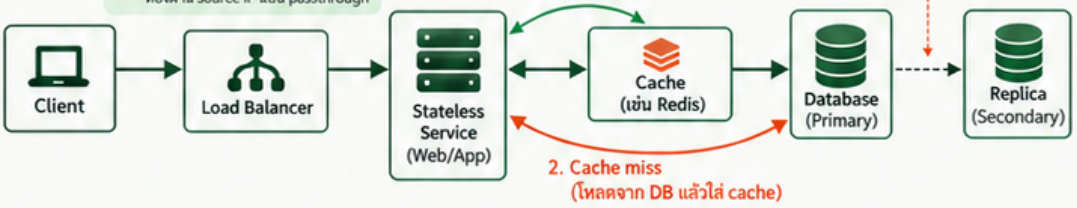
2 สถาปัตยกรรมระบบและการหาคอขวด

แนวทางการออกแบบ

1. เริ่มจาก **requirement** ทั้ง functional และ non-functional ที่วัดได้ และจดสมมติฐาน
2. **วิเคราะห์คอขวดก่อน (bottleneck)** เพราะการเพิ่ม web server ไม่ช่วย เมื่อ database คือคอขวด
3. **Scale out ต้องมี stateless service** เพื่อให้ขยายจำนวน instance ได้

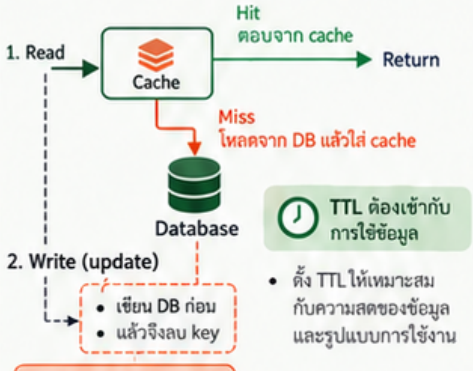
สถาปัตยกรรมตัวอย่าง

L7: route ตาม path/host/header ของ HTTP
L4: สำหรับ TCP/UDP หรือเมื่อต้องผ่าน source IP แบบ passthrough



3 ข้อมูล: Cache, Replica และ Sharding

Cache-aside pattern



TTL ต้องเข้ากับการใช้ข้อมูล
- ตั้ง TTL ให้เหมาะสมกับความสดของข้อมูลและรูปแบบการใช้งาน

Read/Write กับ Replica

Asynchronous replication (ค่าเริ่มต้นที่พบบ่อย)



⚠ ผู้ที่เห็นข้อมูลอาจอ่านไม่เห็นของตัวเอง เพราะข้อมูลอาจยังไม่ถูกคัดลอกไป replica

Synchronous replication

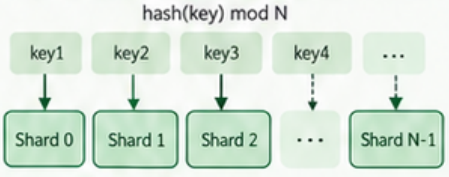


- ลดโอกาสการหายของข้อมูล
- แต่ commit ช้าลง

Sharding (การแบ่งข้อมูล)

หลักการเลือก shard key
- กระจายโหลดให้สมดุล
- ให้ query หลักอยู่ใน shard เดียว

ตัวอย่างการกระจายด้วย hash mod N



⚠ **เมื่อ N เปลี่ยน**
จะพ้องย้าย key ส่วนใหญ่ (เพราะผลลัพธ์ของ mod เปลี่ยน)

4 คิว (Queue) และการประมวลผลงานแบบอะซิงโครนัส

บทบาทของ Queue



- รองรับ peak ได้
- แต่ไม่ให้ความเร็วในการให้บริการ (ไม่เพิ่ม service throughput)

สิ่งที่ต้องออกแบบให้ถูกต้อง

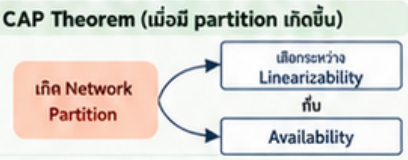
At-least-once delivery
ต้องใช้ idempotent consumer เพื่อให้ประมวลผลลำได้อย่างปลอดภัย

ลำดับของข้อความ
รับประกันลำดับเฉพาะใน partition
ไม่รับประกันลำดับข้าม partition

เหมาะสำหรับงานประมวลผลพื้นหลัง
เช่น ส่งอีเมล, ประมวลผลไฟล์

5 ความทนทานของระบบ: CAP และ PACELC

CAP Theorem (เมื่อมี partition เกิดขึ้น)



เมื่อเกิด network partition ต้องเลือกระหว่าง linearizability กับ availability

PACELC (เวลาปกติและเมื่อมี)

P (มี partition)	E (ไม่มี partition)
ต้องเลือกระหว่าง Consistency กับ Availability	ต้องเลือกระหว่าง Latency กับ Consistency

ภาพรวมคือประเด็นสำคัญ · อ่านรายละเอียดครบในบทเรียน Dev Trail